

Backend Development

The **frontend** is the part of a website/app that users interact with, while the **backend** handles the logic that makes things work.

Example: Login System

When you log in to a website, you see messages like "Login successful" or "Incorrect password." The backend processes this by:

1. Sending your login credentials to the server.
2. Matching them with the stored database records.
3. Sending a response to the frontend based on whether the credentials match or not.

MERN Stack

- **R (React)**: Frontend
 - **M (MongoDB), E (Express.js), N (Node.js)**: Backend
-

Node.js

Node.js is a runtime environment that allows JavaScript to run outside the browser.

History of Node.js

Initially, JavaScript was designed to run only inside browsers. Google developed the **V8 engine** for Chrome, making JavaScript execution faster. In **2009**, **Ryan Dahl** used the V8 engine to create **Node.js**, enabling JavaScript to run on servers. Before Node.js, backend development required languages like **Java**, **PHP**, **Python**, etc. With Node.js, JavaScript became a full-stack language.

JavaScript in Browser vs Node.js

Browser Environment

- Has a **window object** with properties like `alert()`, `prompt()`, etc.
- Variables declared globally are attached to window.

```
let a = 20;
```

```
console.log(window.a); // Output: 20
```

Node.js Environment

- Does **not** have a window object.
 - Uses **global object** instead of window.
 - Cannot use alert(), prompt(), etc.
 - Supports **server-side functionalities**.
-

Non-blocking I/O Model in Node.js

What is I/O?

I/O (Input/Output) refers to operations like reading/writing databases, files, or making HTTP requests. Some I/O tasks execute **instantly** (like logging to the console), while others take **variable time** (like fetching database records).

Blocking vs Non-blocking I/O

```
let user1 = getUserFromDb("user1"); // Takes 5 sec
```

```
console.log(user1);
```

```
let user2 = getUserFromDb("user2"); // Takes 2 sec
```

```
console.log(user2);
```

- **Blocking I/O:** Executes line by line. The second request will only start after the first completes (7 sec total).
 - **Non-blocking I/O:** Both requests execute in parallel, completing in **5 sec** instead of 7.
-

NPM (Node Package Manager)

NPM is the world's largest ecosystem of open-source libraries. It allows developers to:

Use third-party libraries.

Create & publish their own packages.

Running a Node.js Program

1. Create a .js file.
2. Write JavaScript code.
3. Open the terminal, navigate to the file's directory.
4. Run:

```
node filename.js
```

Modules in Node.js

Node.js supports modular programming through:

- **module.exports** (for exporting functions)
- **require()** (for importing functions)

Example: Exporting & Importing Functions

math.js (Module File)

```
function add(a, b) { return a + b; }  
  
function subtract(a, b) { return a - b; }  
  
module.exports = { add, subtract };
```

app.js (Main File)

```
const math = require('./math');  
  
console.log(math.add(5, 3)); // Output: 8
```

Built-in Node.js Modules

Built-in modules do not require installation. To list all built-in modules:

```
console.log(require("module").builtinModules);
```

Path Module in Node.js

The **path module** helps handle file and directory paths in a platform-independent way.

Importing the Path Module

```
const path = require('path');
```

Key Path Methods

1. path.join([...paths])

Joins multiple path segments.

```
const fullPath = path.join('users', 'documents', 'file.txt');  
console.log(fullPath);  
  
// Output: users/documents/file.txt (Unix)  
//      users\documents\file.txt (Windows)
```

2. path.resolve([...paths])

Generates an absolute path.

```
const absolutePath = path.resolve('documents', 'file.txt');  
console.log(absolutePath);  
  
// Output: /absolute/path/to/documents/file.txt
```

3. path.normalize(path)

Cleans up redundant separators.

```
const messyPath = "document\\abc\\..\\text.txt";  
console.log(path.normalize(messyPath));  
  
// Output: document/text.txt
```

4. path.basename(path, [ext])

Extracts the filename from a path.

```
const filePath = '/home/user/index.html';  
console.log(path.basename(filePath)); // Output: index.html  
console.log(path.basename(filePath, '.html')); // Output: index
```

5. path.dirname(path)

Extracts the directory name.

```
console.log(path.dirname('/home/user/index.html'));  
  
// Output: /home/user
```

6. path.extname(path)

Extracts the file extension.

```
console.log(path.extname('/home/user/index.html'));  
  
// Output: .html
```

7. path.parse(path)

Breaks down a path into an object.

```
console.log(path.parse('/home/user/file.txt'));  
  
/* Output:  
  
{  
  root: '/',  
  dir: '/home/user',  
  base: 'file.txt',  
  name: 'file',  
  ext: '.txt'  
}  
*/
```

8. path.format(pathObject)

Converts a parsed path object back into a string.

```
const pathObject = { dir: '/home/user', base: 'file.txt' };  
console.log(path.format(pathObject));  
// Output: /home/user/file.txt
```

9. path.isAbsolute(path)

Checks if a path is absolute.

```
console.log(path.isAbsolute('/home/user/file.txt')); // true  
console.log(path.isAbsolute('file.txt')); // false
```

10. path.relative(from, to)

Finds the relative path from one directory to another.

```
console.log(path.relative('/home/user', '/home/user/file.txt'));  
// Output: file.txt
```

Platform-Specific Properties

path.sep

Returns the path separator:

```
console.log(`Path separator: ${path.sep}`);  
// Output: '/' (Unix) or '\\' (Windows)
```

path.delimiter

Returns the environment variable delimiter:

```
console.log(`Path delimiter: ${path.delimiter}`);  
// Output: ':' (Unix) or ';' (Windows)
```